Experiment 11: CNN-Based Image Classification

Aim:
To build a Convolutional Neural Network (CNN) for image classification using convolution, pooling, and fully connected layers.

Algorithm:

1. Load the image dataset and preprocess the images.
2. Normalize pixel values and prepare class labels.
3. Build a CNN using convolutional and pooling layers.
4. Add fully connected layers for classification.
5. Compile and train the CNN.
6. Evaluate the model using test images.
7. Display predicted classes and analyse accuracy.

Code:

```
#11) CNN for Image Classification


import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
np.random.seed(42)
tf.random.set_seed(42)
# Task 1: Load CIFAR-10 and normalise
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
y_train = y_train.ravel()
y_test = y_test.ravel()
# Use a smaller subset for practical execution
x_train = x_train[:10000].astype("float32") / 255.0
```

```
y_train = y_train[:10000]
x_test = x_test[:2000].astype("float32") / 255.0
y_test = y_test[:2000]
x_fit, x_val = x_train[:8000], x_train[8000:]
y_fit, y_val = y_train[:8000], y_train[8000:]
print("Training images:", x_fit.shape)
print("Validation images:", x_val.shape)
print("Test images:", x_test.shape)
# Task 2: Build CNN
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu",
                           padding="same", name="conv1"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
                           padding="same"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
# Task 3: Train
history = model.fit(
    x_fit, y_fit,
    validation_data=(x_val, y_val),
```

```
    epochs=8,
    batch_size=64,
    verbose=1
)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("CNN Accuracy")
plt.xlabel("Epoch")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("CNN Loss")
plt.xlabel("Epoch")
plt.legend()
plt.tight_layout()
plt.show()
# Task 4: Evaluate and confusion matrix
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_acc * 100, 2), "%")
pred = np.argmax(model.predict(x_test, verbose=0), axis=1)
cm = confusion_matrix(y_test, pred, labels=np.arange(10))
ConfusionMatrixDisplay(
    cm, display_labels=class_names
).plot(xticks_rotation=45, cmap="Blues")
plt.title("CIFAR-10 Confusion Matrix")
plt.tight_layout()
```

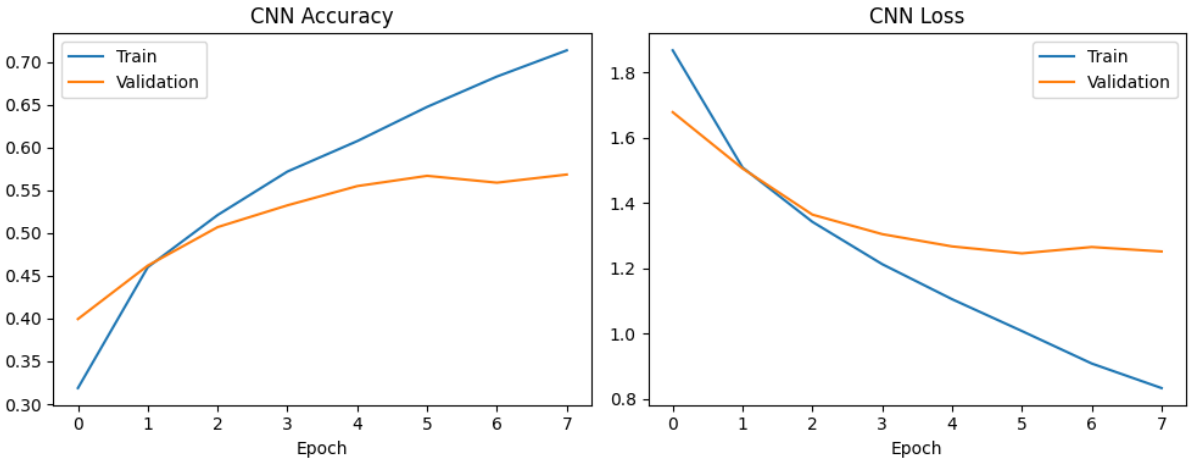
```
plt.show()
# Show up to six misclassified images
wrong = np.where(pred != y_test)[0][:6]
if len(wrong) > 0:
    plt.figure(figsize=(12, 6))
    for j, idx in enumerate(wrong):
        plt.subplot(2, 3, j + 1)
        plt.imshow(x_test[idx])
        plt.title(
            "True: " + class_names[y_test[idx]] +
            "\nPred: " + class_names[pred[idx]]
        )
        plt.axis("off")
    plt.tight_layout()
    plt.show()
else:
    print("No misclassified images in this subset.")
# Task 5: Visualise first convolution layer feature maps
feature_model = tf.keras.Model(
    inputs=model.inputs,
    outputs=model.get_layer("conv1").output
)
features = feature_model.predict(x_test[:1], verbose=0)
plt.figure(figsize=(10, 5))
for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(features[0, :, :, i], cmap="viridis")
    plt.axis("off")
    plt.title("Map " + str(i + 1))
plt.suptitle("First Convolution Layer Feature Maps")
```

```
plt.tight_layout()
plt.show()
```

Output:

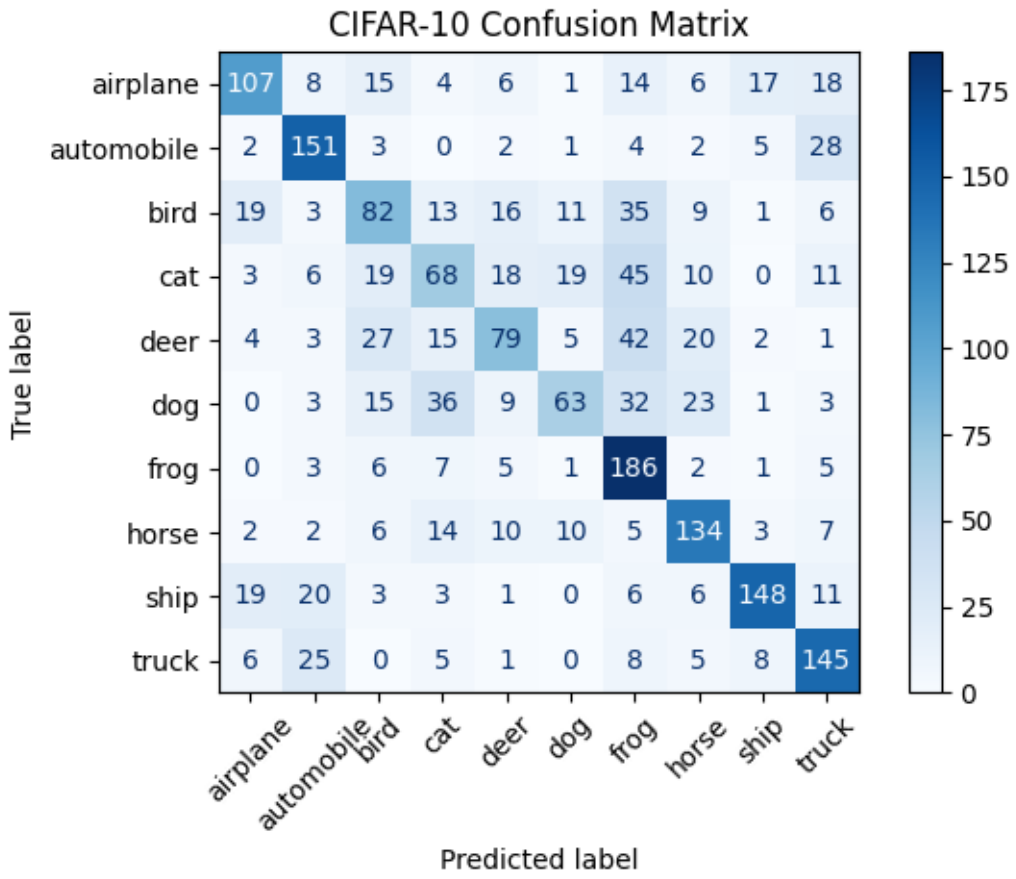
```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 907s 5us/step
Training images: (8000, 32, 32, 3)
Validation images: (2000, 32, 32, 3)
Test images: (2000, 32, 32, 3)
Epoch 1/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 10s 68ms/step - accuracy: 0.3187 - loss: 1.8679 - val_accuracy: 0.3995 - val_loss: 1.6784
Epoch 2/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 8s 61ms/step - accuracy: 0.4601 - loss: 1.5090 - val_accuracy: 0.4620 - val_loss: 1.5059
Epoch 3/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 11s 64ms/step - accuracy: 0.5211 - loss: 1.3425 - val_accuracy: 0.5070 - val_loss: 1.3646
Epoch 4/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 9s 70ms/step - accuracy: 0.5720 - loss: 1.2128 - val_accuracy: 0.5325 - val_loss: 1.3048
Epoch 5/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 9s 69ms/step - accuracy: 0.6076 - loss: 1.1055 - val_accuracy: 0.5550 - val_loss: 1.2672
Epoch 6/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6476 - loss: 1.0080 - val_accuracy: 0.5670 - val_loss: 1.2459
Epoch 7/8
125/125 ━━━━━━━━━━━━━━━━━━━━ 18s 130ms/step - accuracy: 0.6831 - loss: 0.9085 - val_accuracy: 0.5590 - val_loss: 1.2654
Epoch 8/8
```

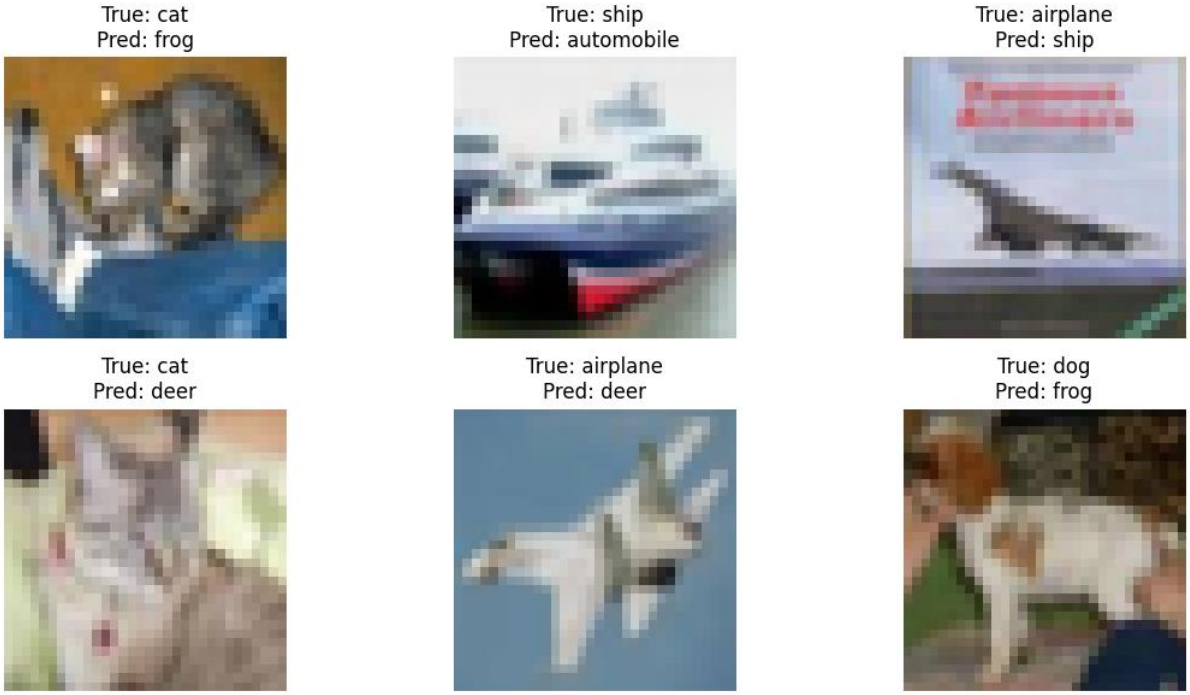
125/125 ━━━━━━━━━━━━━━━━━━━━ **13s** 72ms/step - accuracy: 0.7138 - loss: 0.8332 - val_accuracy: 0.5685 - val_loss: 1.2517

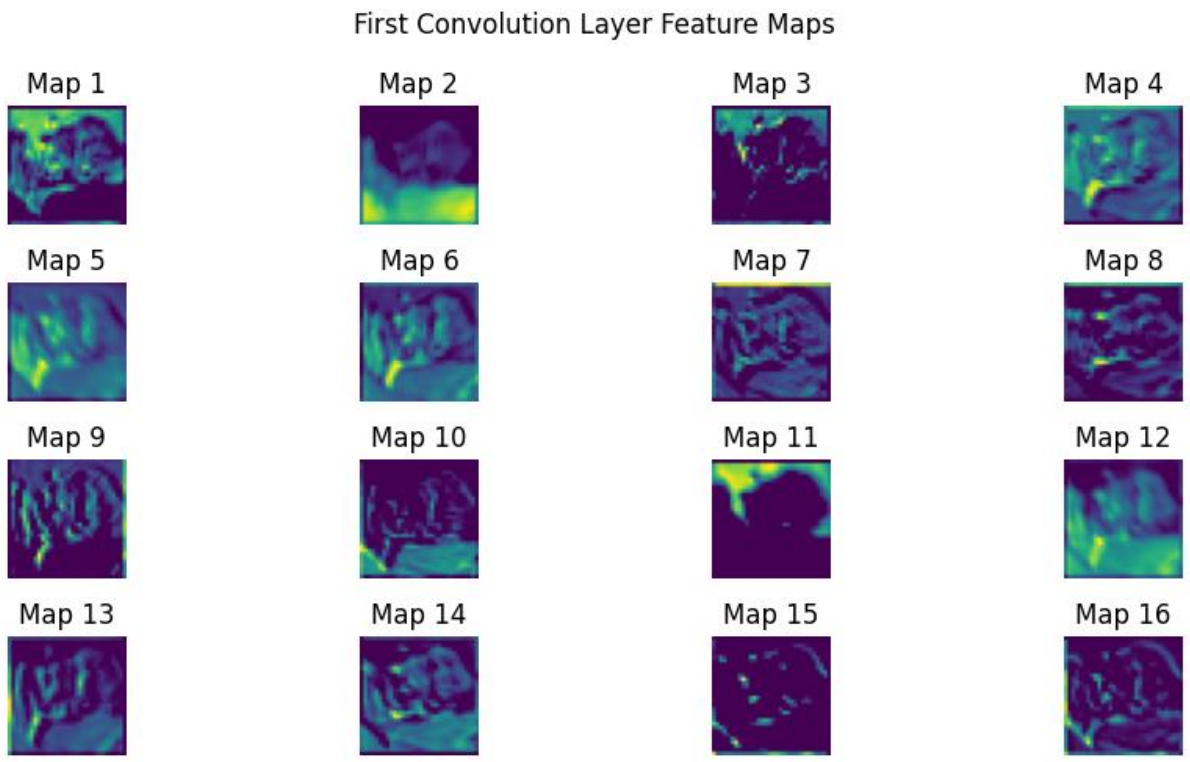


Test Loss: 1.2729

Test Accuracy: 58.15 %







Result:

The CNN model was trained successfully to classify images, and its performance was evaluated using test data.

Experiment 12: CNN Architecture and Hyperparameter Comparison

Aim:
To compare different CNN architectures and hyperparameters to understand their effects on image classification accuracy, training time, and loss.

Algorithm:

1. Prepare and preprocess the image dataset.
2. Define CNN models with different architectures.
3. Change hyperparameters such as filters, kernel size, learning rate, and batch size.
4. Train each model under the selected settings.
5. Record accuracy, loss, and training time.
6. Compare the results using tables or graphs.
7. Identify the most suitable configuration.

Code:

```
#12) CNN Architecture and Hyperparameter Comparison


import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
np.random.seed(42)
tf.random.set_seed(42)
# Task 1: Load and prepare CIFAR-10
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
x_train = x_train[:10000].astype("float32") / 255.0
y_train = y_train[:10000].ravel()
x_test = x_test[:2000].astype("float32") / 255.0
y_test = y_test[:2000].ravel()
x_fit, x_val = x_train[:8000], x_train[8000:]
y_fit, y_val = y_train[:8000], y_train[8000:]
```

```
# Configurations vary depth, filters, kernel, pooling,
# batch normalisation, learning rate and batch size.
configs = [
    {"depth": 2, "filters": 32, "kernel": 3,
     "pool": "max", "bn": False, "lr": 0.001, "batch": 64},
    {"depth": 3, "filters": 32, "kernel": 3,
     "pool": "max", "bn": True, "lr": 0.001, "batch": 64},
    {"depth": 4, "filters": 32, "kernel": 3,
     "pool": "max", "bn": True, "lr": 0.001, "batch": 64},
    {"depth": 2, "filters": 32, "kernel": 5,
     "pool": "average", "bn": False, "lr": 0.001, "batch": 64},
    {"depth": 3, "filters": 64, "kernel": 3,
     "pool": "average", "bn": True, "lr": 0.0005, "batch": 32},
    {"depth": 4, "filters": 64, "kernel": 5,
     "pool": "max", "bn": True, "lr": 0.0005, "batch": 32}
]
def build_cnn(cfg):
    layers = [tf.keras.layers.Input(shape=(32, 32, 3))]
    for i in range(cfg["depth"]):
        filters = cfg["filters"] * min(2 ** i, 4)
        layers.append(tf.keras.layers.Conv2D(
            filters, (cfg["kernel"], cfg["kernel"]),
            padding="same", use_bias=not cfg["bn"]
        ))
        if cfg["bn"]:
            layers.append(tf.keras.layers.BatchNormalization())
        layers.append(tf.keras.layers.Activation("relu"))
        if i < 2:
            if cfg["pool"] == "max":
                layers.append(tf.keras.layers.MaxPooling2D(2))
```

```
            else:
                layers.append(tf.keras.layers.AveragePooling2D(2))
    layers.extend([
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation="softmax")
    ])
    model = tf.keras.Sequential(layers)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=cfg["lr"]
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Tasks 1-4: Train and compare configurations
results = []
for i, cfg in enumerate(configs):
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(42)
    model = build_cnn(cfg)
    start = time.perf_counter()
    history = model.fit(
        x_fit, y_fit,
        validation_data=(x_val, y_val),
        epochs=5,
        batch_size=cfg["batch"],
        verbose=0
    )
    elapsed = time.perf_counter() - start
```

```
    test_loss, test_acc = model.evaluate(
        x_test, y_test, verbose=0
    )
    results.append({
        "Config": i + 1,
        "Conv Layers": cfg["depth"],
        "Filters": cfg["filters"],
        "Kernel": str(cfg["kernel"]) + "x" + str(cfg["kernel"]),
        "Pooling": cfg["pool"],
        "Batch Norm": cfg["bn"],
        "Learning Rate": cfg["lr"],
        "Batch Size": cfg["batch"],
        "Test Accuracy (%)": round(test_acc * 100, 2),
        "Parameters": model.count_params(),
        "Training Time (s)": round(elapsed, 2)
    })
    print(
        "Configuration", i + 1,
        "| Accuracy:", round(test_acc * 100, 2), "%",
        "| Parameters:", model.count_params(),
        "| Time:", round(elapsed, 2), "seconds"
    )
# Task 5: Summary table
results_df = pd.DataFrame(results)
print("\nCNN Configuration Comparison")
print(results_df.to_string(index=False))
# Accuracy comparison
plt.figure(figsize=(9, 5))
plt.bar(
    results_df["Config"].astype(str),
```

```
    results_df["Test Accuracy (%)"]
)
plt.xlabel("Configuration")
plt.ylabel("Test Accuracy (%)")
plt.title("CNN Architecture Accuracy Comparison")
plt.grid(axis="y")
plt.show()
# Parameter count and training time
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].bar(
    results_df["Config"].astype(str),
    results_df["Parameters"]
)
axes[0].set_title("Model Size")
axes[0].set_xlabel("Configuration")
axes[0].set_ylabel("Number of Parameters")
axes[1].bar(
    results_df["Config"].astype(str),
    results_df["Training Time (s)"]
)
axes[1].set_title("Training Time")
axes[1].set_xlabel("Configuration")
axes[1].set_ylabel("Seconds")
plt.tight_layout()
plt.show()
```

Output:

Configuration 1 | Accuracy: 31.95 % | Parameters: 20042 | Time: 46.28 seconds

Configuration 2 | Accuracy: 39.5 % | Parameters: 95210 | Time: 97.81 seconds

Configuration 3 | Accuracy: 40.35 % | Parameters: 243178 | Time: 151.67 seconds

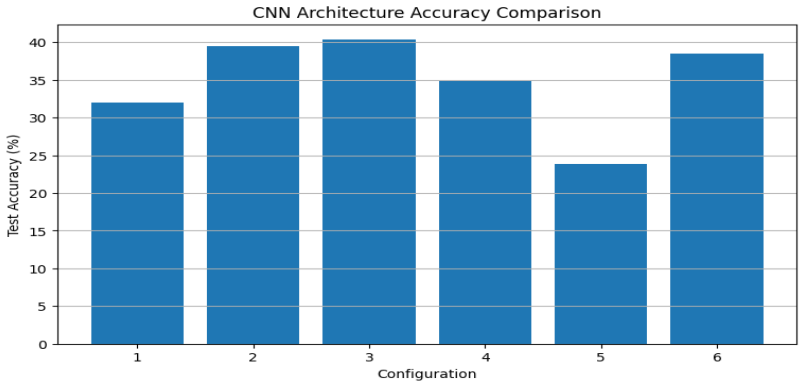
Configuration 4 | Accuracy: 34.9 % | Parameters: 54346 | Time: 97.66 seconds

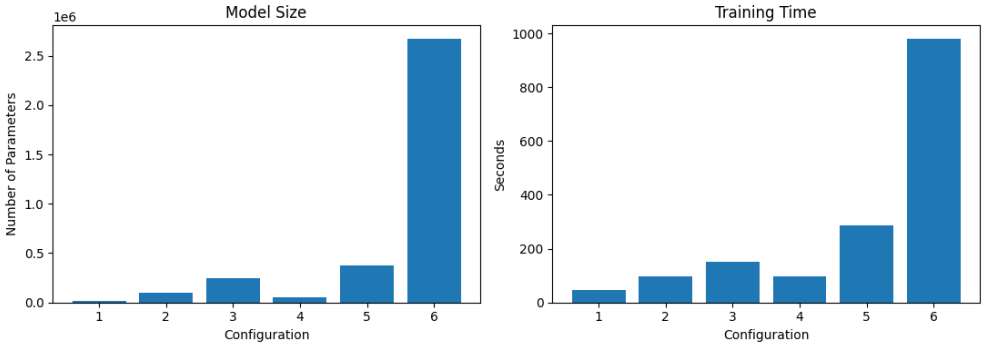
Configuration 5 | Accuracy: 23.85 % | Parameters: 374730 | Time: 288.11 seconds

Configuration 6 | Accuracy: 38.5 % | Parameters: 2672586 | Time: 980.6 seconds

CNN Configuration Comparison

Config	Conv Layers	Filters	Kernel	Pooling	Batch Norm	Learning Rate	Batch Size	Test Accuracy (%)	Parameters	Training Time (s)
1	2	32	3x3	max	False	0.0010	64	31.95	20042	46.28
2	3	32	3x3	max	True	0.0010	64	39.50	95210	97.81
3	4	32	3x3	max	True	0.0010	64	40.35	243178	151.67
4	2	32	5x5	average	False	0.0010	64	34.90	54346	97.66
5	3	64	3x3	average	True	0.0005	32	23.85	374730	288.11
6	4	64	5x5	max	True	0.0005	32	38.50	2672586	980.60





Result:

Different CNN architectures and hyperparameters were compared, and their effects on classification performance and training efficiency were analysed.

Experiment 13: Sequential Dataset Preparation and RNN Prediction

Aim:
To prepare sequential time-series data by handling missing values, normalizing values, and creating sliding windows, then use an RNN to predict electricity demand.

Algorithm:

1. Load or generate hourly electricity-demand data.
2. Handle missing values and normalize the data.
3. Create input sequences using the previous 24 hours.
4. Split the data chronologically into training and testing sets.
5. Build and train a SimpleRNN model.
6. Predict electricity demand for the test period.
7. Calculate MAE and RMSE and plot actual versus predicted values.

Code:

```
#13)


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import time
np.random.seed(42)
tf.random.set_seed(42)
# 1. Create one year of hourly electricity-demand data
dates = pd.date_range("2025-01-01", periods=365*24, freq="h")
t = np.arange(len(dates))
demand = (
    500
    + 80 * np.sin(2 * np.pi * t / 24)
    + 40 * np.sin(2 * np.pi * t / (24 * 7))
    + 20 * np.sin(2 * np.pi * t / (24 * 365))
```

```
    + np.random.normal(0, 10, len(t)))
df = pd.DataFrame({"Date": dates, "Demand": demand})
# Insert missing values for demonstration
df.loc[[100, 500, 1000], "Demand"] = np.nan
df["Demand"] = df["Demand"].interpolate().bfill().ffill()
print("Dataset shape:", df.shape)
print("Missing values:", df["Demand"].isna().sum())
print(df.head())
# 2. Chronological split and scaling
values = df["Demand"].to_numpy(dtype=np.float32)
split = int(len(values) * 0.80)
train_min = values[:split].min()
train_max = values[:split].max()
scaled = (values - train_min) / (train_max - train_min + 1e-8)
# Create 24-hour input windows to predict the next hour
window = 24
X, y, target_indices = [], [], []
for i in range(window, len(scaled)):
    X.append(scaled[i-window:i])
    y.append(scaled[i])
    target_indices.append(i)
X = np.array(X, dtype=np.float32).reshape(-1, window, 1)
y = np.array(y, dtype=np.float32)
target_indices = np.array(target_indices)
train_mask = target_indices < split
test_mask = target_indices >= split
X_train, y_train = X[train_mask], y[train_mask]
X_test, y_test = X[test_mask], y[test_mask]
print("\nTraining sequences:", X_train.shape)
print("Testing sequences:", X_test.shape)
```

```
# 3. Build SimpleRNN
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(window, 1)),
    tf.keras.layers.SimpleRNN(32, activation="tanh"),
    tf.keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")
# 4. Train the model
start = time.time()
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    shuffle=False,
    verbose=1)
print("Training time:", round(time.time() - start, 2), "seconds")
# 5. Predict and calculate metrics
pred_scaled = model.predict(X_test, verbose=0).ravel()
actual = y_test * (train_max - train_min) + train_min
predicted = pred_scaled * (train_max - train_min) + train_min
mae = np.mean(np.abs(actual - predicted))
rmse = np.sqrt(np.mean((actual - predicted) ** 2))
print("\nEVALUATION")
print("MAE:", round(float(mae), 4))
print("RMSE:", round(float(rmse), 4))
# Show sample predictions
results = pd.DataFrame({
    "Actual Demand": actual[:10],
    "Predicted Demand": predicted[:10]})
print("\nFirst 10 predictions:")
```

```
print(results.round(2).to_string(index=False))
# Plot actual and predicted demand
plt.figure(figsize=(12, 5))
plt.plot(actual[:200], label="Actual demand")
plt.plot(predicted[:200], label="Predicted demand")
plt.xlabel("Test hour")
plt.ylabel("Electricity demand")
plt.title("Experiment 13: Actual vs Predicted Demand")
plt.legend()
plt.grid(True)
plt.show()
# Plot training and validation loss
plt.figure(figsize=(7, 4))
plt.plot(history.history["loss"], label="Training loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("MSE loss")
plt.title("SimpleRNN Training")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Dataset shape: (8760, 2)
Missing values: 0
                 Date      Demand
0 2025-01-01 00:00:00  504.967142
1 2025-01-01 01:00:00  520.832874
2 2025-01-01 02:00:00  549.494779
3 2025-01-01 03:00:00  576.320456
4 2025-01-01 04:00:00  572.959570
```

Training sequences: (6984, 24, 1)

Testing sequences: (1752, 24, 1)

Epoch 1/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **6s** 15ms/step - loss: 0.0107 - val_loss: 0.0021

Epoch 2/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **5s** 13ms/step - loss: 0.0020 - val_loss: 0.0018

Epoch 3/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **4s** 18ms/step - loss: 0.0017 - val_loss: 0.0017

Epoch 4/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 12ms/step - loss: 0.0017 - val_loss: 0.0017

Epoch 5/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 10ms/step - loss: 0.0016 - val_loss: 0.0016

Epoch 6/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 8ms/step - loss: 0.0016 - val_loss: 0.0016

Epoch 7/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **1s** 5ms/step - loss: 0.0016 - val_loss: 0.0016

Epoch 8/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **1s** 5ms/step - loss: 0.0016 - val_loss: 0.0016

Epoch 9/10

197/197 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **1s** 5ms/step - loss: 0.0015 - val_loss: 0.0016

Epoch 10/10

197/197 ━━━━━━━━━━━━━━━━━━━━ **1s** 5ms/step - loss: 0.0015 - val_loss: 0.0016

Training time: 25.25 seconds

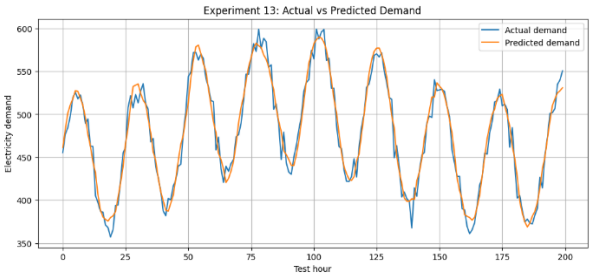
EVALUATION

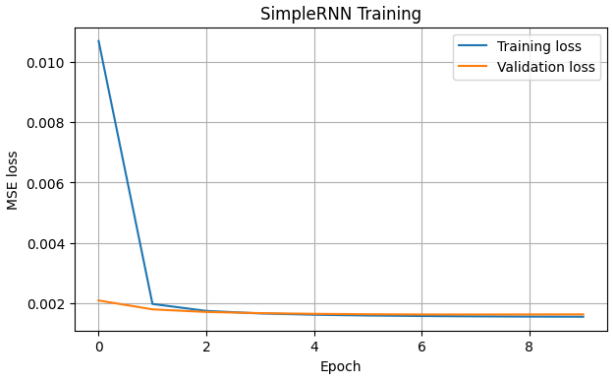
MAE: 10.2065

RMSE: 12.7442

First 10 predictions:

Actual Demand	Predicted Demand
455.450012	461.239990
476.820007	482.440002
484.350006	501.209991
498.130005	511.010010
519.119995	517.890015
525.619995	527.380005
517.909973	526.650024
522.359985	518.799988
510.059998	510.320007
489.299988	496.459991





Result:
The time-series data was prepared successfully, and the SimpleRNN model was used to predict electricity demand and evaluate prediction errors.

Experiment 14: Unfolded RNN Architecture Visualization

Aim:
To visualize an unfolded Recurrent Neural Network (RNN), calculate hidden states for a five-word sentence, and verify manual calculations against a Keras RNN.

Algorithm:

1. Prepare a five-word sentence and convert the words into numerical vectors.
2. Define an RNN with three hidden units.
3. Extract the input, recurrent, and output weights.
4. Calculate the hidden state at each time step manually.
5. Compare the manual hidden states with the Keras outputs.
6. Draw the unfolded RNN architecture.
7. Plot the hidden-state values.

Code:

#14)

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import networkx as nx
np.random.seed(42)
tf.random.set_seed(42)
# Five words represented as five one-hot vectors
words = ["We", "love", "deep", "learning", "today"]
X = np.eye(5, dtype=np.float32).reshape(1, 5, 5)
print("Input sentence:", " ".join(words))
```

```
print("Input tensor shape:", X.shape)
# Build the RNN
inputs = tf.keras.Input(shape=(5, 5))
rnn_layer = tf.keras.layers.SimpleRNN(
    3, activation="tanh", return_sequences=True, name="rnn")
hidden_output = rnn_layer(inputs)
output_layer = tf.keras.layers.Dense(1, name="output")
outputs = output_layer(hidden_output)
model = tf.keras.Model(inputs, outputs)
keras_hidden = rnn_layer(X, training=False).numpy()[0]
keras_output = output_layer(keras_hidden).numpy().ravel()
# 1. Extract weights
kernel, recurrent_kernel, bias = rnn_layer.get_weights()
W_hy, b_y = output_layer.get_weights()
print("\nWEIGHT SHAPES")
print("Input weights W_xh:", kernel.shape)
print("Recurrent weights W_hh:", recurrent_kernel.shape)
print("Output weights W_hy:", W_hy.shape)
# 2. Manually calculate hidden states
manual_hidden = []
h = np.zeros(3, dtype=np.float32)
for t in range(5):
    x_t = X[0, t, :]
    h = np.tanh(x_t @ kernel + h @ recurrent_kernel + bias)
    manual_hidden.append(h.copy())
manual_hidden = np.array(manual_hidden)
manual_output = (manual_hidden @ W_hy + b_y).ravel()
print("\nMANUAL HIDDEN STATES")
print(np.round(manual_hidden, 4))
print("\nKERAS HIDDEN STATES")
```

```
print(np.round(keras_hidden, 4))
difference = np.max(np.abs(manual_hidden - keras_hidden))
print("\nMaximum hidden-state difference:", round(float(difference), 7))
print("Manual calculation matches Keras:", np.allclose(
    manual_hidden, keras_hidden, atol=1e-5))
print("\nOutput values:", np.round(manual_output, 4))
# 3. Draw the unfolded network
G = nx.DiGraph()
positions = {}
for t in range(5):
    G.add_node(f"x{t+1}", layer="input")
    G.add_node(f"h{t+1}", layer="hidden")
    G.add_node(f"y{t+1}", layer="output")
    positions[f"x{t+1}"] = (t, 2)
    positions[f"h{t+1}"] = (t, 1)
    positions[f"y{t+1}"] = (t, 0)
    G.add_edge(f"x{t+1}", f"h{t+1}", label="W_xh")
    G.add_edge(f"h{t+1}", f"y{t+1}", label="W_hy")
    if t > 0:
        G.add_edge(f"h{t}", f"h{t+1}", label="W_hh")
colors = []
for node in G.nodes:
    if node.startswith("x"):
        colors.append("lightblue")
    elif node.startswith("h"):
        colors.append("lightgreen")
    else:
        colors.append("lightyellow")
plt.figure(figsize=(13, 5))
nx.draw_networkx(
```

```
    G, positions, with_labels=True, node_color=colors,
    node_size=1300, font_size=10, arrows=True)
edge_labels = nx.get_edge_attributes(G, "label")
nx.draw_networkx_edge_labels(
    G, positions, edge_labels=edge_labels, font_size=8)
plt.title("Experiment 14: RNN Unfolded Across Five Time Steps")
plt.axis("off")
plt.show()
# 4. Plot hidden-state values
plt.figure(figsize=(8, 4))
for i in range(3):
    plt.plot(range(1, 6), manual_hidden[:, i],
             marker="o", label=f"Hidden unit {i+1}")
plt.xlabel("Time step")
plt.ylabel("Hidden-state value")
plt.title("Hidden States Across Time")
plt.xticks(range(1, 6))
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Input sentence: We love deep learning today
Input tensor shape: (1, 5, 5)

WEIGHT SHAPES
Input weights W_xh: (5, 3)
Recurrent weights W_hh: (3, 3)
Output weights W_hy: (3, 1)

MANUAL HIDDEN STATES
```

[[0.3806 -0.2294 -0.5632]

 [0.1232 0.2271 0.4282]

 [-0.2649 -0.6048 -0.6705]

 [0.1287 0.2331 -0.4148]

 [0.4625 0.7728 0.5458]]

KERAS HIDDEN STATES

[[0.3806 -0.2294 -0.5632]

 [0.1232 0.2271 0.4282]

 [-0.2649 -0.6048 -0.6705]

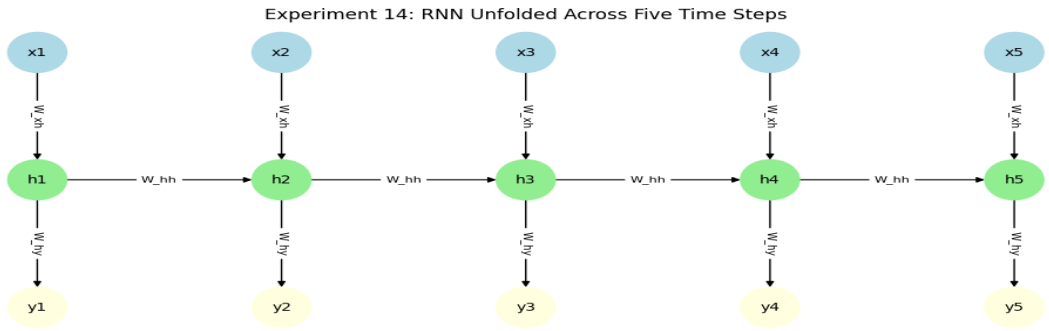
 [0.1287 0.2331 -0.4148]

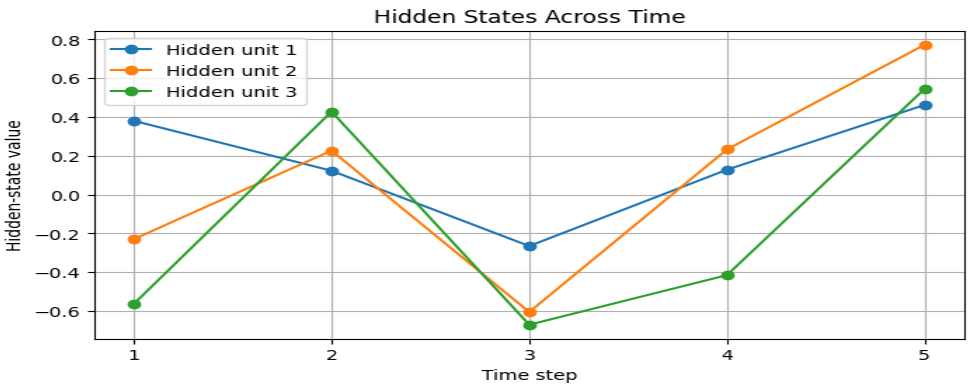
 [0.4625 0.7728 0.5458]]

Maximum hidden-state difference: 1e-07

Manual calculation matches Keras: True

Output values: [0.2638 -0.169   0.068   0.5184 0.1619]





Result:
The unfolded RNN architecture was visualized, and manually calculated hidden states were compared with the Keras model outputs.

Experiment 15: Backpropagation Through Time (BPTT) and Gradient Analysis

Aim:
To study Backpropagation Through Time (BPTT) by analysing gradient behaviour for different sequence lengths and activation functions.

Algorithm:

1. Prepare sequential input data.
2. Define recurrent computations using tanh and ReLU activation functions.
3. Calculate the output loss.
4. Use gradient calculations to measure the contribution of earlier time steps.
5. Repeat the experiment with sequence lengths of 10, 50, and 100.
6. Compare gradient magnitudes for both activation functions.
7. Plot the gradient behaviour and identify vanishing or exploding gradients.

Code:

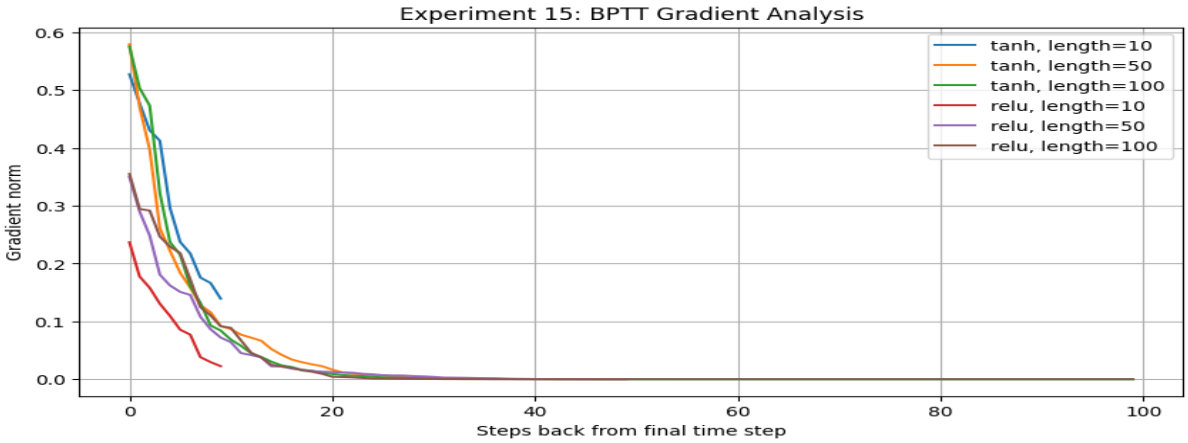
```
#15)


import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import pandas as pd
np.random.seed(42)
tf.random.set_seed(42)
def analyse_gradients(length, activation):
    units = 16
    inputs = tf.random.normal((1, length, 4), stddev=0.5)
```

```
    # Build a Keras SimpleRNN and initialise its weights
    layer = tf.keras.layers.SimpleRNN(
        units, activation=activation, return_sequences=True)
    layer(inputs)
    kernel, recurrent_kernel, bias = layer.get_weights()
    activation_fn = tf.keras.activations.get(activation)
    states = []
    # Unroll the same SimpleRNN recurrence under GradientTape
    with tf.GradientTape(persistent=True) as tape:
        h = tf.zeros((1, units))
        for t in range(length):
            h = activation_fn(
                tf.matmul(inputs[:, t, :], kernel)
                + tf.matmul(h, recurrent_kernel)
                + bias)
            tape.watch(h)
            states.append(h)
        prediction = tf.reduce_mean(h)
        loss = tf.square(prediction - 1.0)
    gradients = tape.gradient(loss, states)
    del tape
    norms = np.array([
        float(tf.norm(g).numpy()) if g is not None else 0.0
        for g in gradients])
    return norms, float(loss.numpy())
lengths = [10, 50, 100]
activations = ["tanh", "relu"]
records = []
plt.figure(figsize=(10, 5))
for activation in activations:
```

```
    for length in lengths:
        norms, loss = analyse_gradients(length, activation)
        # Distance from the final time step
        distance = np.arange(length - 1, -1, -1)
        plt.plot(
            distance, norms,
            label=f"{activation}, length={length}"
        )
        records.append({
            "Activation": activation,
            "Sequence length": length,
            "Initial-time gradient norm": norms[0],
            "Final-time gradient norm": norms[-1],
            "Loss": loss})
plt.xlabel("Steps back from final time step")
plt.ylabel("Gradient norm")
plt.title("Experiment 15: BPTT Gradient Analysis")
plt.legend()
plt.grid(True)
plt.show()
table = pd.DataFrame(records)
print("\nGRADIENT ANALYSIS RESULTS")
print(table.round(6).to_string(index=False))
print("\nINTERPRETATION")
print("Very small earlier gradients indicate possible vanishing gradients.")
print("Very large gradient norms indicate possible exploding gradients.")
print("Compare the curves for tanh and ReLU; results depend on initial weights.")
```

Output:



GRADIENT ANALYSIS RESULTS

Activation	Sequence length	Initial-time gradient norm	Final-time gradient norm	Loss
tanh	10	0.139282	0.527637	1.113602
tanh	50	0.000066	0.579690	1.344161
tanh	100	0.000000	0.575575	1.325145
relu	10	0.022600	0.236969	0.224617
relu	50	0.000024	0.350189	0.490531
relu	100	0.000000	0.355214	0.504709

INTERPRETATION

Very small earlier gradients indicate possible vanishing gradients.

Very large gradient norms indicate possible exploding gradients.

Compare the curves for tanh and ReLU; results depend on initial weights.

Result:
Gradient behaviour was analysed for different sequence lengths and activation functions, demonstrating how gradients may vanish or grow during recurrent training.

Experiment 16: Comparison of RNN and LSTM

Aim:
To implement and compare SimpleRNN and LSTM models for daily order prediction using the same input sequence length and hidden-unit configuration.

Algorithm:

1. Prepare daily order data and normalize the values.
2. Create sequences using the previous 30 days.
3. Split the data into training and testing sets.
4. Build and train SimpleRNN and LSTM models.
5. Record training time and loss for both models.
6. Calculate MAE, RMSE, and MAPE.
7. Compare the actual and predicted order values using plots and tables.

Code:

#16)

```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import time
np.random.seed(42)
tf.random.set_seed(42)
```

```
# 1. Generate two years of daily order counts
days = 730
t = np.arange(days)
weekly = 20 * np.sin(2 * np.pi * t / 7)
seasonal = 35 * np.sin(2 * np.pi * t / 365)
festival = np.zeros(days)
for day in [100, 250, 360, 500, 620, 700]:
    festival[day: min(day + 5, days)] += 50
orders = (
    200 + weekly + seasonal + festival
    + np.random.normal(0, 8, days))
orders = np.maximum(orders, 1).astype(np.float32)
df = pd.DataFrame({
    "Day": pd.date_range("2024-01-01", periods=days, freq="D"),
    "Orders": orders})
print("Dataset shape:", df.shape)
print(df.head())
# 2. Scale using the training period only
lookback = 30
train_end = int(days * 0.70)
val_end = int(days * 0.85)
minimum = orders[:train_end].min()
maximum = orders[:train_end].max()
scaled = (orders - minimum) / (maximum - minimum + 1e-8)
X, y, target_day = [], [], []
for i in range(lookback, days):
    X.append(scaled[i-lookback:i])
    y.append(scaled[i])
    target_day.append(i)
X = np.array(X, dtype=np.float32).reshape(-1, lookback, 1)
```

```
y = np.array(y, dtype=np.float32)
target_day = np.array(target_day)
train_mask = target_day < train_end
val_mask = (target_day >= train_end) & (target_day < val_end)
test_mask = target_day >= val_end
X_train, y_train = X[train_mask], y[train_mask]
X_val, y_val = X[val_mask], y[val_mask]
X_test, y_test = X[test_mask], y[test_mask]
print("\nTraining sequences:", X_train.shape)
print("Validation sequences:", X_val.shape)
print("Testing sequences:", X_test.shape)
# 3. Build models with the same number of units
def build_model(cell_type):
    if cell_type == "SimpleRNN":
        recurrent = tf.keras.layers.SimpleRNN(32)
    else:
        recurrent = tf.keras.layers.LSTM(32)
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(lookback, 1)),
        recurrent,
        tf.keras.layers.Dense(1)])
    model.compile(optimizer="adam", loss="mse")
    return model
results = []
histories = {}
predictions = {}
for cell_type in ["SimpleRNN", "LSTM"]:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = build_model(cell_type)
```

```
    start = time.time()
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=10,
        batch_size=32,
        shuffle=False,
        verbose=1)
    elapsed = time.time() - start
    pred_scaled = model.predict(X_test, verbose=0).ravel()
    actual = y_test * (maximum - minimum) + minimum
    predicted = pred_scaled * (maximum - minimum) + minimum
    mae = np.mean(np.abs(actual - predicted))
    rmse = np.sqrt(np.mean((actual - predicted) ** 2))
    mape = np.mean(
        np.abs((actual - predicted) / np.maximum(np.abs(actual), 1e-8))
    ) * 100
    results.append({
        "Model": cell_type,
        "Final train loss": history.history["loss"][-1],
        "Final validation loss": history.history["val_loss"][-1],
        "MAE": mae,
        "RMSE": rmse,
        "MAPE (%)": mape,
        "Training time (s)": elapsed,
        "Parameters": model.count_params()})
    histories[cell_type] = history.history
    predictions[cell_type] = predicted
# 4. Display comparison table
results_df = pd.DataFrame(results)
```

```
print("\nMODEL COMPARISON")
print(results_df.round(4).to_string(index=False))
# 5. Plot training and validation loss
plt.figure(figsize=(9, 5))
for name, history in histories.items():
    plt.plot(history["loss"], label=f"{name} train")
    plt.plot(history["val_loss"], linestyle="--",
             label=f"{name} validation")
plt.xlabel("Epoch")
plt.ylabel("MSE loss")
plt.title("Experiment 16: Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
# Plot actual versus predicted orders
actual = y_test * (maximum - minimum) + minimum
plt.figure(figsize=(11, 5))
plt.plot(actual, label="Actual orders", linewidth=2)
for name, pred in predictions.items():
    plt.plot(pred, label=f"{name} prediction")
plt.xlabel("Test day")
plt.ylabel("Order count")
plt.title("Actual vs Predicted Daily Orders")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Dataset shape: (730, 2)
        Day      Orders
0 2024-01-01  203.973709
```

1 2024-01-02 215.132980

2 2024-01-03 225.884827

3 2024-01-04 222.668594

4 2024-01-05 191.85717

Training sequences: (480, 30, 1)

Validation sequences: (110, 30, 1)

Testing sequences: (110, 30, 1)

Epoch 1/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **4s** 66ms/step - loss: 0.1094 - val_loss: 0.0452

Epoch 2/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 19ms/step - loss: 0.0470 - val_loss: 0.0096

Epoch 3/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - loss: 0.0268 - val_loss: 0.0170

Epoch 4/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - loss: 0.0179 - val_loss: 0.0097

Epoch 5/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - loss: 0.0148 - val_loss: 0.0094

Epoch 6/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - loss: 0.0122 - val_loss: 0.0085

Epoch 7/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - loss: 0.0113 - val_loss: 0.0077

Epoch 8/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - loss: 0.0106 - val_loss: 0.0068

Epoch 9/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - loss: 0.0101 - val_loss: 0.0060

Epoch 10/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - loss: 0.0095 - val_loss: 0.0057

Epoch 1/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **2s** 30ms/step - loss: 0.1708 - val_loss: 0.0240

Epoch 2/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0318 - val_loss: 0.0302

Epoch 3/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0196 - val_loss: 0.0115

Epoch 4/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - loss: 0.0180 - val_loss: 0.0160

Epoch 5/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0173 - val_loss: 0.0131

Epoch 6/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0164 - val_loss: 0.0133

Epoch 7/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0162 - val_loss: 0.0129

Epoch 8/10

15/15 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0158 - val_loss: 0.0126

Epoch 9/10

15/15 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0156 - val_loss: 0.0124

Epoch 10/10

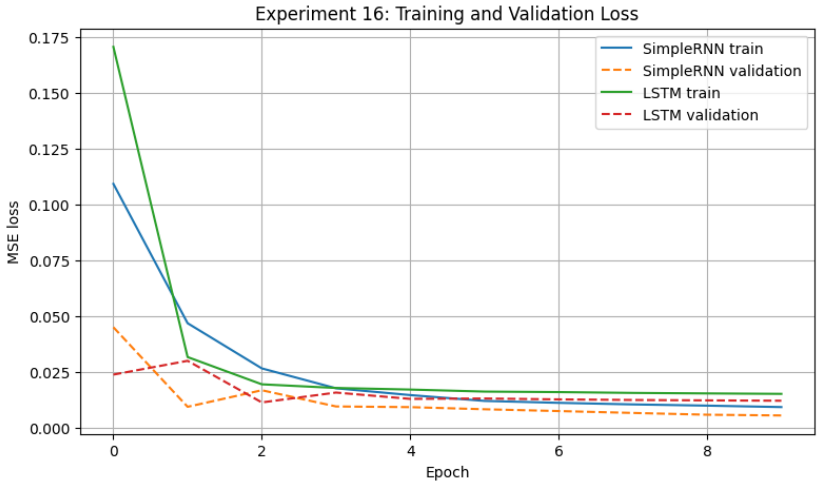
15/15 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - loss: 0.0154 - val_loss: 0.0123

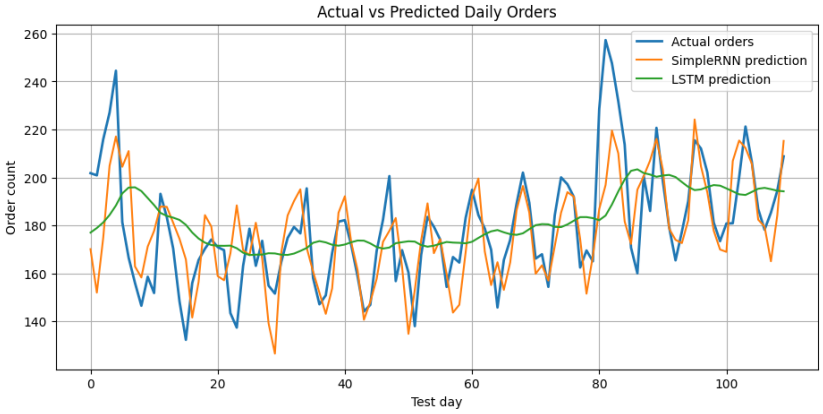
WARNING:tensorflow:5 out of the last 60 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d33ed33aca0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:6 out of the last 63 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d33ed33aca0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

MODEL COMPARISON

Model	Final train loss	Final validation loss	MAE	RMSE	MAPE (%)	Training time (s)	Parameters
SimpleRNN	0.0095	0.0057	13.6732	18.113501	7.6829	5.5216	1121
LSTM	0.0154	0.0123	17.8190	22.669300	10.0635	3.6112	4385





Result:
SimpleRNN and LSTM models were trained and compared using prediction errors, training time, and forecast performance.

Experiment 17: Gradient Clipping in Recurrent Neural Networks

Aim:
To implement gradient clipping using clipnorm and clipvalue and compare their effects with an unclipped model during recurrent neural network training.

Algorithm:

1. Prepare text data and convert characters into numerical values.
2. Build a character-level recurrent neural network.
3. Train the model without gradient clipping.
4. Train separate models using clipnorm and clipvalue.
5. Record gradient magnitudes and training losses.

6. Compare the loss curves and gradient behaviour.
7. Analyse how clipping helps control excessively large gradients.

Code:

```
#17)


import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
np.random.seed(42)
tf.random.set_seed(42)
# 1. Prepare a character-level text corpus
text = (
    "the market changes every day and prices move up and down. "
    "good models learn patterns from data. "
) * 100
vocab = sorted(set(text))
char_to_id = {ch: i for i, ch in enumerate(vocab)}
encoded = np.array([char_to_id[ch] for ch in text], dtype=np.int32)
sequence_length = 20
X, Y = [], []
for i in range(0, len(encoded) - sequence_length, 3):
    X.append(encoded[i:i + sequence_length])
    Y.append(encoded[i + 1:i + sequence_length + 1])
X = np.array(X, dtype=np.int32)
Y = np.array(Y, dtype=np.int32)
print("Corpus characters:", len(encoded))
print("Vocabulary size:", len(vocab))
print("Input sequences:", X.shape)
print("Target sequences:", Y.shape)
# 2. Build a character-level RNN
```

```
def build_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(sequence_length,)),
        tf.keras.layers.Embedding(len(vocab), 16),
        tf.keras.layers.SimpleRNN(32, return_sequences=True),
        tf.keras.layers.Dense(len(vocab))])
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(
    from_logits=True)
# Different clipping settings
settings = [
    ("No clipping", None, None),
    ("Clipnorm 1.0", 1.0, None),
    ("Clipnorm 5.0", 5.0, None),
    ("Clipvalue 0.5", None, 0.5),
    ("Clipvalue 1.0", None, 1.0)]
all_results = {}
def train_setting(name, clipnorm, clipvalue):
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = build_model()
    optimizer = tf.keras.optimizers.SGD(
        learning_rate=0.5,
        clipnorm=clipnorm,
        clipvalue=clipvalue)
    losses = []
    norms = []
    spike_steps = []
    nan_step = None
    batch_size = 32
    steps = 100
```

```
for step in range(steps):
    indices = np.random.randint(0, len(X), batch_size)
    xb = tf.convert_to_tensor(X[indices])
    yb = tf.convert_to_tensor(Y[indices])
    with tf.GradientTape() as tape:
        logits = model(xb, training=True)
        loss = loss_fn(yb, logits)
    gradients = tape.gradient(loss, model.trainable_variables)
    global_norm = tf.linalg.global_norm(gradients)
    loss_value = float(loss.numpy())
    norm_value = float(global_norm.numpy())
    # Detect non-finite values before updating weights
    if not np.isfinite(loss_value) or not np.isfinite(norm_value):
        nan_step = step + 1
        print(name, "- non-finite loss/gradient at step", nan_step)
        break
    losses.append(loss_value)
    norms.append(norm_value)
    if norm_value > 100:
        spike_steps.append(step + 1)
    optimizer.apply_gradients(
        zip(gradients, model.trainable_variables))
if nan_step is None:
    print(name, "- completed all", steps, "steps")
print("  Maximum raw gradient norm:",
      round(max(norms), 4) if norms else "No finite gradients")
print("  Spike steps (norm > 100):",
      spike_steps if spike_steps else "None detected")
print("  Non-finite step:", nan_step if nan_step else "None")
print("  Final recorded loss:",
```

```
        round(losses[-1], 4) if losses else "Unavailable")
    return {
        "losses": losses,
        "norms": norms,
        "spikes": spike_steps,
        "nan_step": nan_step}
# 3. Train with each setting
for name, clipnorm, clipvalue in settings:
    print("\nTraining:", name)
    all_results[name] = train_setting(
        name, clipnorm, clipvalue)
# 4. Plot gradient norms
plt.figure(figsize=(11, 5))
for name, result in all_results.items():
    if result["norms"]:
        plt.plot(result["norms"], label=name)
plt.xlabel("Training step")
plt.ylabel("Global gradient norm before clipping")
plt.title("Experiment 17: Gradient Norm Comparison")
plt.legend()
plt.grid(True)
plt.show()
# 5. Plot losses
plt.figure(figsize=(11, 5))
for name, result in all_results.items():
    if result["losses"]:
        plt.plot(result["losses"], label=name)
plt.xlabel("Training step")
plt.ylabel("Loss")
plt.title("Experiment 17: Loss Comparison")
```

```
plt.legend()
plt.grid(True)
plt.show()
print("\nRECOMMENDATION")
print("Start with clipnorm=1.0 as a practical baseline.")
print("Compare stability and loss before choosing the final threshold.")
```

Output:

```
Corpus characters: 9600
Vocabulary size: 23
Input sequences: (3194, 20)
Target sequences: (3194, 20)
Training: No clipping
No clipping - completed all 100 steps
  Maximum raw gradient norm: 0.8944
  Spike steps (norm > 100): None detected
  Non-finite step: None
  Final recorded loss: 0.4222
Training: Clipnorm 1.0
Clipnorm 1.0 - completed all 100 steps
  Maximum raw gradient norm: 0.9887
  Spike steps (norm > 100): None detected
  Non-finite step: None
  Final recorded loss: 0.5001
Training: Clipnorm 5.0
Clipnorm 5.0 - completed all 100 steps
  Maximum raw gradient norm: 1.1324
  Spike steps (norm > 100): None detected
  Non-finite step: None
  Final recorded loss: 0.3709
Training: Clipvalue 0.5
```

Clipvalue 0.5 - completed all 100 steps

Maximum raw gradient norm: 0.7518

Spike steps (norm > 100): None detected

Non-finite step: None

Final recorded loss: 0.4439

Training: Clipvalue 1.0

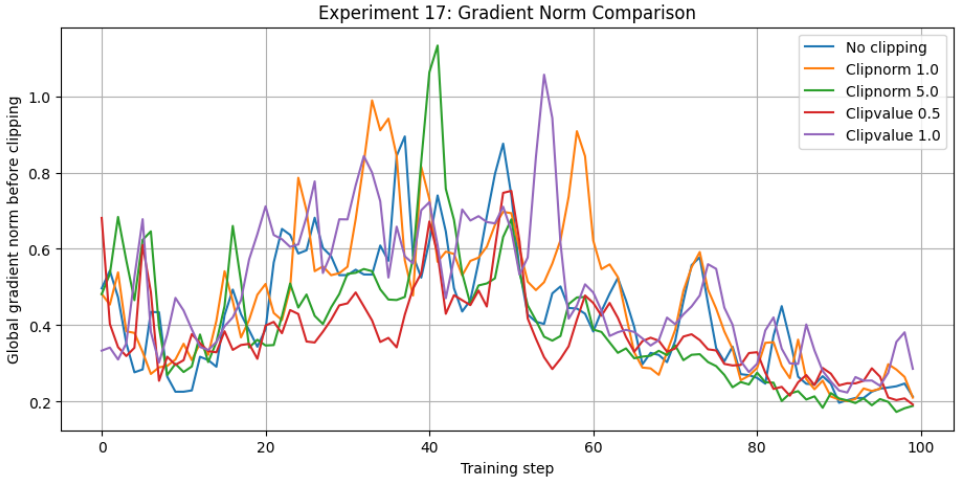
Clipvalue 1.0 - completed all 100 steps

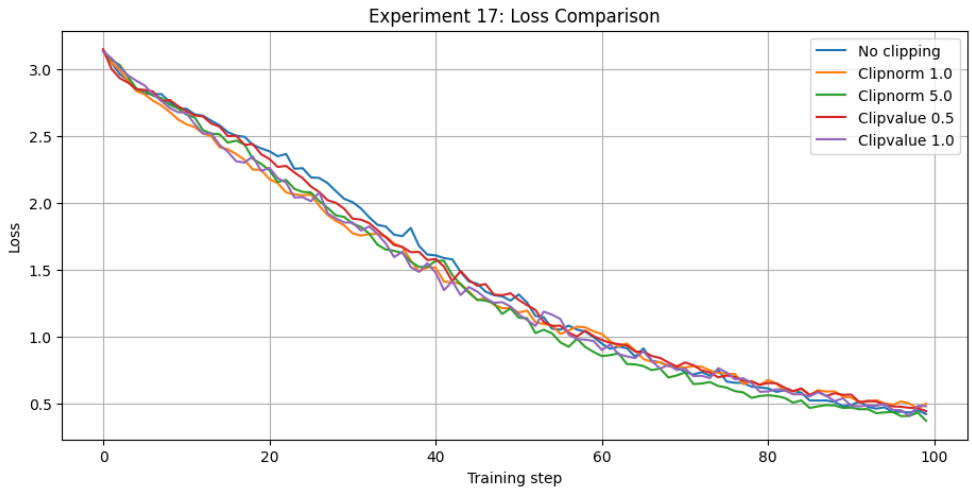
Maximum raw gradient norm: 1.056

Spike steps (norm > 100): None detected

Non-finite step: None

Final recorded loss: 0.4809





RECOMMENDATION

Start with clipnorm=1.0 as a practical baseline.

Result:
Gradient clipping methods were implemented and compared with training without clipping to study their effects on gradient stability.

Experiment 18: RNN, LSTM and GRU Hyperparameter Tuning

Aim:
To build and compare RNN, LSTM, and GRU models using hyperparameter tuning for sentiment classification of IMDB movie reviews.

Algorithm:

1. Load the IMDB review dataset and preprocess the text.
2. Convert reviews into sequences and pad them to a fixed length.
3. Define RNN, LSTM, and GRU model architectures.
4. Select hyperparameters such as units, learning rate, and dropout.
5. Use KerasTuner to test different configurations.

6. Train and evaluate the best configuration for each model.
7. Compare validation accuracy and loss.

Code:

#18)

```
import numpy as np
import pandas as pd
import tensorflow as tf
import keras_tuner as kt
import matplotlib.pyplot as plt
import time
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping
np.random.seed(42)
tf.random.set_seed(42)
# 1. Load and prepare IMDB sentiment data
vocab_size = 10000
max_length = 120
(x_train_all, y_train_all), (x_test_all, y_test_all) = (
    imdb.load_data(num_words=vocab_size)
)
x_train_all = pad_sequences(
    x_train_all, maxlen=max_length, padding="post", truncating="post"
)
x_test_all = pad_sequences(
    x_test_all, maxlen=max_length, padding="post", truncating="post"
)
# Smaller subsets keep the experiment practical
x_train = x_train_all[:6000]
```

```
y_train = np.asarray(y_train_all[:6000])
x_val = x_train_all[6000:7500]
y_val = np.asarray(y_train_all[6000:7500])
x_test = x_test_all[:3000]
y_test = np.asarray(y_test_all[:3000])
print("Training reviews:", x_train.shape)
print("Validation reviews:", x_val.shape)
print("Testing reviews:", x_test.shape)
print("Padded review length:", max_length)
# 2. Model builder for each recurrent cell type
def build_model(hp, cell_type):
    embedding_dim = hp.Choice("embedding_dim", [16, 32])
    units = hp.Choice("units", [32, 64])
    num_layers = hp.Int("num_layers", min_value=1, max_value=2)
    dropout = hp.Choice("dropout", [0.2, 0.4])
    learning_rate = hp.Choice(
        "learning_rate", [0.001, 0.0005]
    )
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Input(shape=(max_length,)))
    model.add(tf.keras.layers.Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        mask_zero=True
    ))
    for i in range(num_layers):
        return_sequences = i < num_layers - 1

        if cell_type == "SimpleRNN":
            layer = tf.keras.layers.SimpleRNN(
```

```
            units, return_sequences=return_sequences
        )
    elif cell_type == "LSTM":
        layer = tf.keras.layers.LSTM(
            units, return_sequences=return_sequences
        )
    else:
        layer = tf.keras.layers.GRU(
            units, return_sequences=return_sequences
        )
    model.add(layer)
    model.add(tf.keras.layers.Dropout(dropout))
model.add(tf.keras.layers.Dense(1, activation="sigmoid"))
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=learning_rate
    ),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
return model
# 3. Tune each recurrent cell type separately
results = []
histories = {}
for cell_type in ["SimpleRNN", "LSTM", "GRU"]:
    print("\nTuning model:", cell_type)
    tf.keras.backend.clear_session()
    tuner = kt.RandomSearch(
        hypermodel=lambda hp: build_model(hp, cell_type),
        objective="val_accuracy",
```

```
    max_trials=3,
    executions_per_trial=1,
    directory="experiment18_tuning",
    project_name=cell_type,
    overwrite=True,
    seed=42
)
tuner_start = time.time()
tuner.search(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=3,
    batch_size=64,
    callbacks=[
        EarlyStopping(
            monitor="val_loss",
            patience=1,
            restore_best_weights=True
        )
    ],
    verbose=1)
tuning_time = time.time() - tuner_start
best_hp = tuner.get_best_hyperparameters(1)[0]
print("Best hyperparameters:", best_hp.values)
# Retrain the selected configuration for a fair timing comparison
tf.keras.backend.clear_session()
tf.random.set_seed(42)
best_model = build_model(best_hp, cell_type)
# Combine training and validation data for final fitting
x_dev = np.concatenate([x_train, x_val], axis=0)
```

```
    y_dev = np.concatenate([y_train, y_val], axis=0)
    train_start = time.time()
    history = best_model.fit(
        x_dev, y_dev,
        epochs=3,
        batch_size=64,
        verbose=1)
    training_time = time.time() - train_start
    test_loss, test_accuracy = best_model.evaluate(
        x_test, y_test, verbose=0)
    results.append({
        "Model": cell_type,
        "Best units": best_hp.get("units"),
        "Layers": best_hp.get("num_layers"),
        "Embedding dim": best_hp.get("embedding_dim"),
        "Dropout": best_hp.get("dropout"),
        "Learning rate": best_hp.get("learning_rate"),
        "Test accuracy": test_accuracy,
        "Test loss": test_loss,
        "Parameters": best_model.count_params(),
        "Final training time (s)": training_time,
        "Tuning time (s)": tuning_time})
    histories[cell_type] = history.history
# 4. Print comparison table
results_df = pd.DataFrame(results)
print("\nEXPERIMENT 18: MODEL COMPARISON")
print(results_df.round(4).to_string(index=False))
# 5. Plot training accuracy for selected configurations
plt.figure(figsize=(9, 5))
for cell_type, history in histories.items():
```

```
    plt.plot(
        history["accuracy"],
        marker="o",
        label=cell_type)
plt.xlabel("Epoch")
plt.ylabel("Training accuracy")
plt.title("Experiment 18: Selected Model Accuracy")
plt.legend()
plt.grid(True)
plt.show()
# Plot final test accuracy
plt.figure(figsize=(7, 4))
plt.bar(results_df["Model"], results_df["Test accuracy"])
plt.ylabel("Test accuracy")
plt.title("RNN vs LSTM vs GRU")
plt.ylim(0, 1)
plt.grid(axis="y")
plt.show()
```

Output:

Trial 3 Complete [00h 01m 33s]

val_accuracy: 0.7846666574478149

Best val_accuracy So Far: 0.8113333582878113

Total elapsed time: 00h 02m 51s

Best hyperparameters: {'embedding_dim': 32, 'units': 32, 'num_layers': 1, 'dropout': 0.2, 'learning_rate': 0.0005}

Epoch 1/3

118/118 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **13s** 80ms/step - accuracy: 0.5144 - loss: 0.6921

Epoch 2/3

118/118 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **11s** 90ms/step - accuracy: 0.6863 - loss: 0.5936

Epoch 3/3

118/118 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **11s** 93ms/step - accuracy: 0.8575 - loss: 0.3383

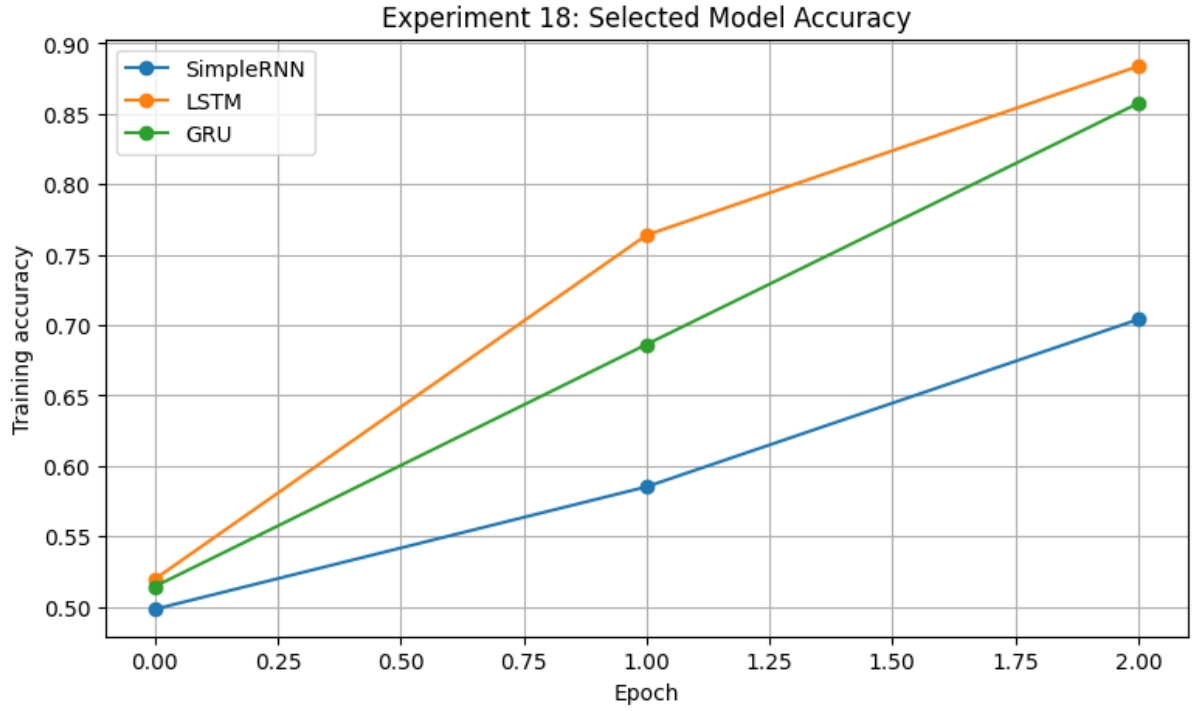
EXPERIMENT 18: MODEL COMPARISON

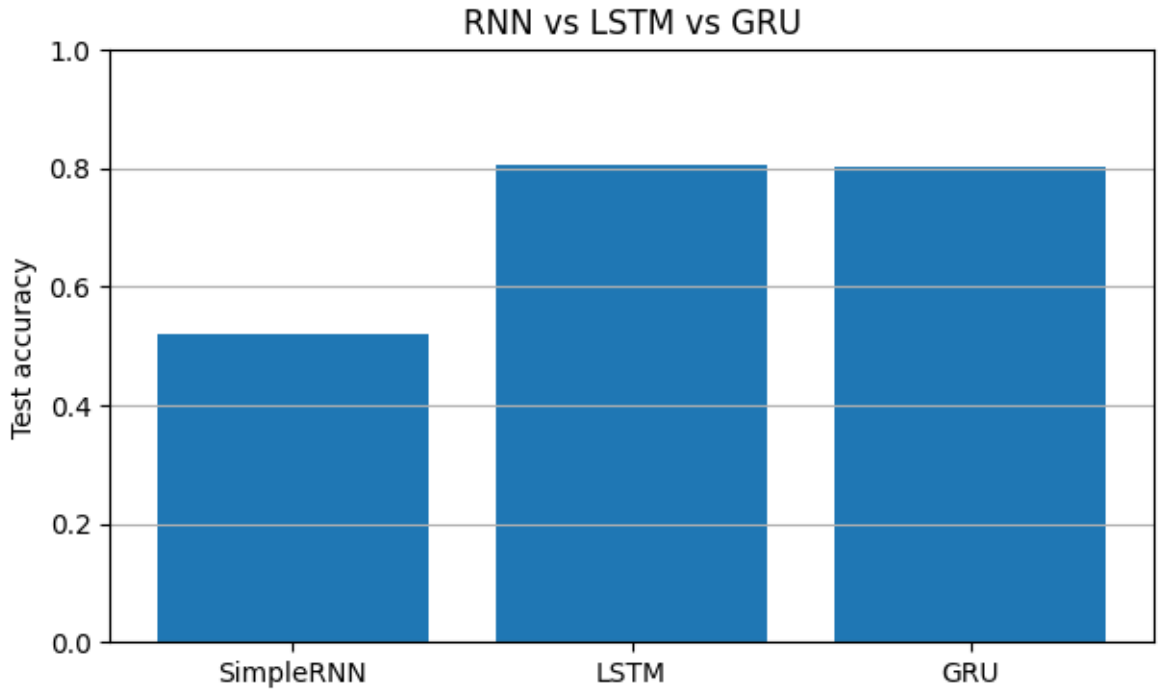
Model Best units Layers Embedding dim Dropout Learning rate Test accuracy Test loss Parameters Final training time (s) Tuning time (s)

SimpleRNN 64 1 16 0.2 0.0010 0.5193 0.7249 165249 24.8188 74.1368

LSTM 32 1 32 0.2 0.0005 0.8057 0.4536 328353 31.3716 193.9580

GRU 32 1 32 0.2 0.0005 0.8023 0.4389 326369 35.2987 171.4056





Result:
RNN, LSTM, and GRU models were tuned and compared for sentiment classification using their validation performance.

Experiment 19: Principal Component Analysis (PCA) and Explained Variance

Aim:
To implement PCA on the Olivetti faces dataset, calculate explained variance, verify the results using Scikit-learn, and study image reconstruction using different numbers of principal components.

Algorithm:

1. Load the Olivetti faces dataset and normalize the image data.
2. Calculate the mean and center the data.
3. Calculate the covariance matrix and its eigenvalues and eigenvectors.
4. Sort the eigenvalues and calculate explained variance.
5. Compare the results with Scikit-learn PCA.

6. Plot the scree plot and cumulative explained variance.
7. Display eigenfaces and reconstruct images using 10, 50, and 100 components.

Code:

#19)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.decomposition import PCA
np.random.seed(42)
# 1. Load face dataset
faces = fetch_olivetti_faces(shuffle=True, random_state=42)
X = faces.data.astype(np.float64)
images = faces.images
print("Dataset shape:", X.shape)
print("Image shape:", images.shape)
# Centre the data
mean_face = X.mean(axis=0)
X_centered = X - mean_face
# 2. PCA from scratch using the dual covariance matrix
# The dual matrix is only 400 x 400, avoiding a 4096 x 4096 matrix.
sample_covariance = (X_centered @ X_centered.T) / (len(X) - 1)
eigenvalues, eigenvectors = np.linalg.eigh(sample_covariance)
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]
# Convert sample-space eigenvectors into pixel-space components
components = X_centered.T @ eigenvectors
components = components / np.maximum(
    np.linalg.norm(components, axis=0), 1e-12)
```

```
# Remove tiny negative eigenvalues caused by numerical precision
eigenvalues = np.maximum(eigenvalues, 0)
explained_variance = eigenvalues
explained_ratio = explained_variance / explained_variance.sum()
cumulative_variance = np.cumsum(explained_ratio)
# 3. Verify against Scikit-learn PCA
sk_pca = PCA(svd_solver="full")
sk_pca.fit(X)
comparison_count = 100
manual_ratio = explained_ratio[:comparison_count]
sklearn_ratio = sk_pca.explained_variance_ratio_[:comparison_count]
print("\nPCA VERIFICATION")
print("First 10 manual variance ratios:")
print(np.round(manual_ratio[:10], 6))
print("First 10 sklearn variance ratios:")
print(np.round(sklearn_ratio[:10], 6))
print("Variance ratios agree:",
    np.allclose(manual_ratio, sklearn_ratio, atol=1e-5))
# 4. Number of components needed for 95% variance
n95 = np.searchsorted(cumulative_variance, 0.95) + 1
print("\nComponents for 95% variance:", n95)
print("Variance retained:", round(float(cumulative_variance[n95-1]), 4))
# Scree plot
plt.figure(figsize=(8, 4))
plt.plot(np.arange(1, 101), explained_ratio[:100], marker=".")
plt.xlabel("Principal component")
plt.ylabel("Explained variance ratio")
plt.title("Scree Plot")
plt.grid(True)
plt.show()
```

```
# Cumulative explained variance
plt.figure(figsize=(8, 4))
plt.plot(np.arange(1, len(cumulative_variance) + 1),
        cumulative_variance)
plt.axhline(0.95, linestyle="--", label="95% variance")
plt.axvline(n95, linestyle="--", label=f"{n95} components")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("Cumulative Explained Variance")
plt.legend()
plt.grid(True)
plt.show()
# 5. Display top eigenfaces
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(components[:, i].reshape(64, 64), cmap="gray")
    ax.set_title(f"PC {i+1}")
    ax.axis("off")
plt.suptitle("Top 10 Eigenfaces")
plt.tight_layout()
plt.show()
# Reconstruct one face with 10, 50 and 100 components
face_index = 0
component_counts = [10, 50, 100]
fig, axes = plt.subplots(1, 4, figsize=(13, 4))
axes[0].imshow(images[face_index], cmap="gray")
axes[0].set_title("Original")
axes[0].axis("off")
for ax, k in zip(axes[1:], component_counts):
    pca_k = PCA(n_components=k, svd_solver="full")
```

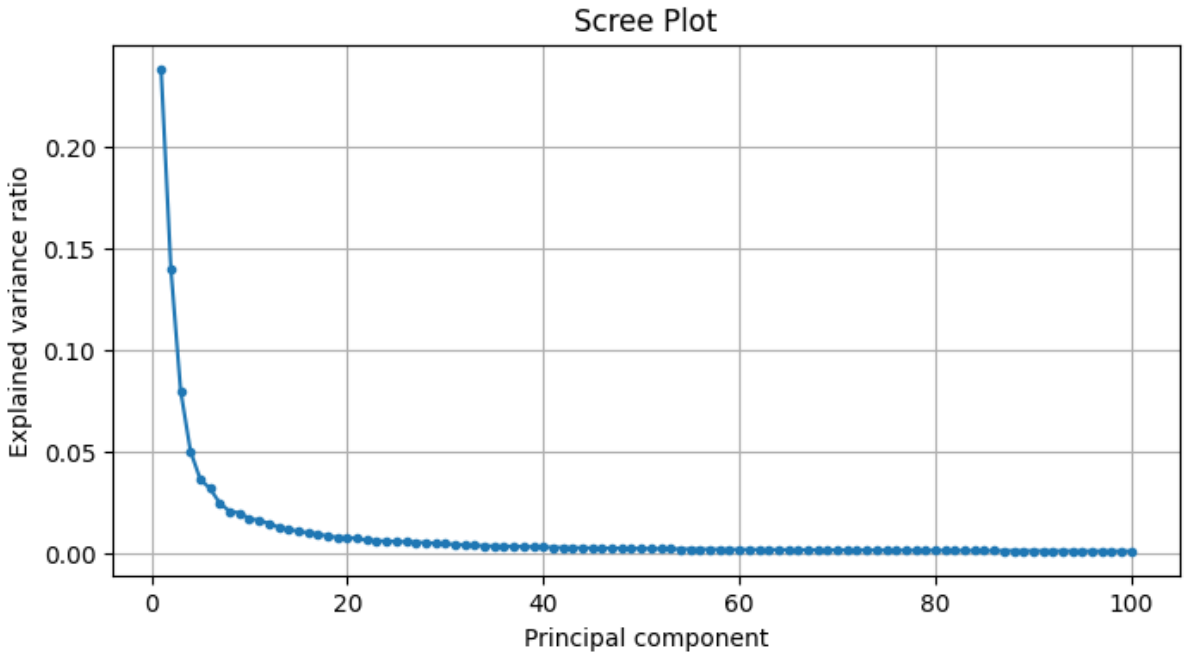
```
    pca_k.fit(X)
    reconstructed = pca_k.inverse_transform(
        pca_k.transform(X[face_index:face_index+1])
    )[0]
    ax.imshow(reconstructed.reshape(64, 64), cmap="gray")
    ax.set_title(f"{k} components")
    ax.axis("off")
plt.suptitle("Face Reconstruction")
plt.tight_layout()
plt.show()
```

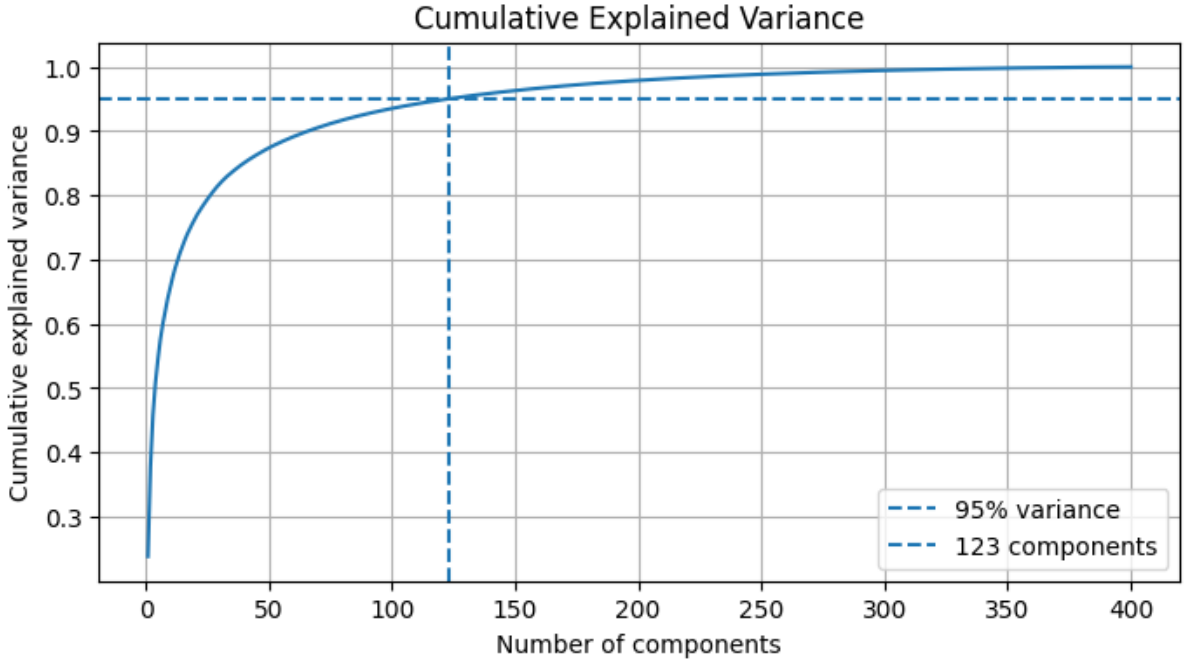
Output:

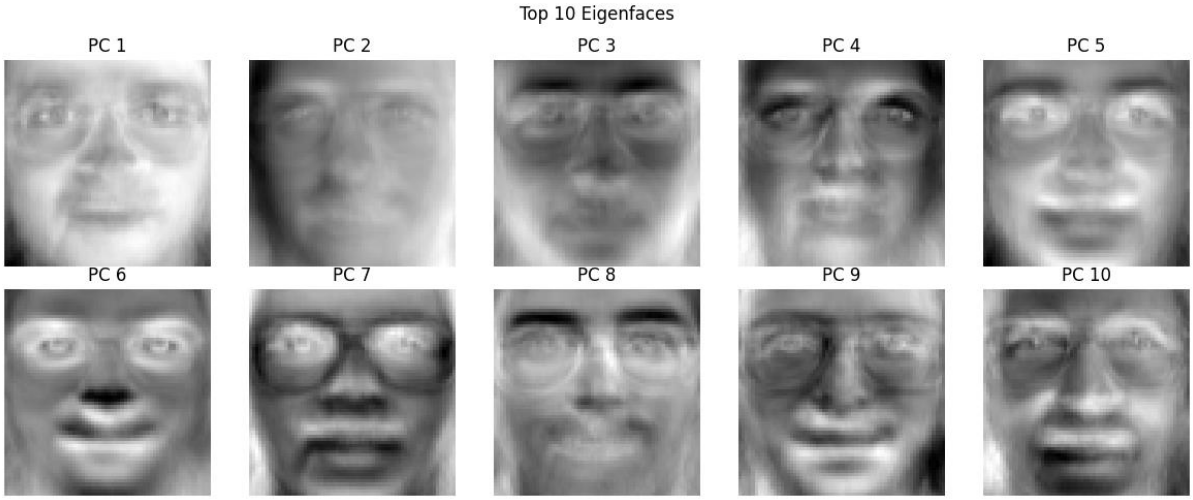
```
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to /root/scikit_learn_data
Dataset shape: (400, 4096)
Image shape: (400, 64, 64)

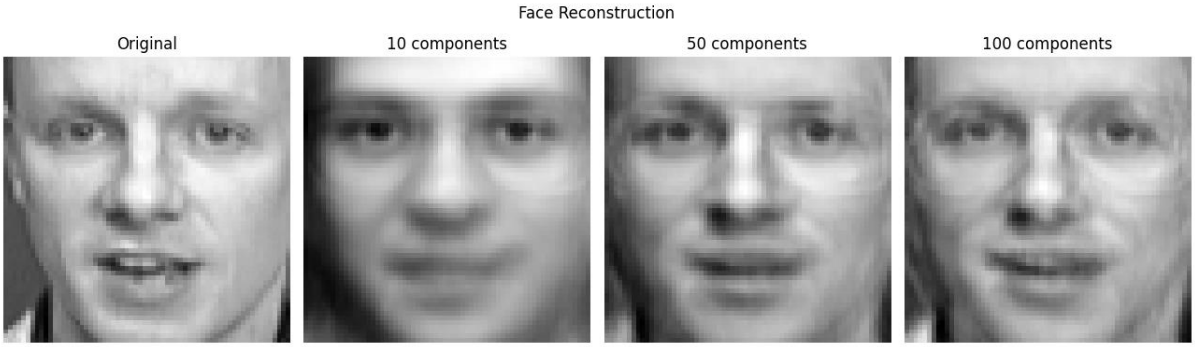
PCA VERIFICATION
First 10 manual variance ratios:
[0.238127 0.13994  0.079686 0.049983 0.036098 0.031569 0.024268 0.020364
 0.019581 0.016721]
First 10 sklearn variance ratios:
[0.238127 0.13994  0.079686 0.049983 0.036098 0.031569 0.024268 0.020364
 0.019581 0.016721]
Variance ratios agree: True

Components for 95% variance: 123
Variance retained: 0.9504
```


Scree Plot
Explained variance ratio
0.20
0.15
0.10
0.05
0.00
0
20
40
60
80
100
Principal component


Cumulative Explained Variance
Cumulative explained variance
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
50
100
150
200
250
300
350
400
Number of components
95% variance
123 components





Result:

PCA was applied successfully to reduce the dimensionality of face images, analyse explained variance, and compare image reconstruction at different component counts.

Experiment 20: Probabilistic Principal Component Analysis (PPCA)

Aim:
To study Probabilistic PCA using synthetic sensor data, estimate noise variance, reconstruct data using different numbers of components, and generate new samples.

Algorithm:

1. Generate synthetic multivariate sensor data using latent variables and noise.
2. Introduce missing values and replace them using suitable imputation.
3. Fit a PCA-based model to estimate the principal components and noise variance.
4. Reconstruct the data using different numbers of components.
5. Calculate and compare reconstruction errors.
6. Generate new samples using the estimated latent representation.
7. Plot the reconstruction errors and analyse the results.

Code:

```
#20)


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
np.random.seed(42)
# 1. Generate correlated sensor readings
n_samples = 600
n_features = 5
latent = np.random.randn(n_samples, 2)
W_true = np.array([
    [2.0, 0.2],
    [1.5, 0.3],
    [0.1, 1.8],
    [0.4, 1.2],
```

```
    [1.0, -0.8]
])
mu_true = np.array([25, 40, 100, 12, 50])
noise_std = 0.5
X = latent @ W_true.T + mu_true
X += np.random.normal(0, noise_std, X.shape)
columns = ["Temperature", "Vibration", "Pressure", "Current", "Humidity"]
df = pd.DataFrame(X, columns=columns)
# Add occasional missing values and impute column means
df.iloc[5, 1] = np.nan
df.iloc[20, 3] = np.nan
df.iloc[100, 2] = np.nan
imputer = SimpleImputer(strategy="mean")
X_clean = imputer.fit_transform(df)
print("Sensor data shape:", X_clean.shape)
print("Missing values after imputation:",
      int(np.isnan(X_clean).sum()))
print("\nFirst five sensor readings:")
print(pd.DataFrame(X_clean, columns=columns).head().round(2))
# 2. Fit PPCA using sklearn PCA's noise variance estimate
max_components = 5
ppca = PCA(n_components=max_components, svd_solver="full")
ppca.fit(X_clean)
sigma2 = ppca.noise_variance_
print("\nPPCA NOISE ESTIMATE")
print("Estimated noise variance:", round(float(sigma2), 6))
print("True generated noise variance:", noise_std ** 2)
# 3. Compare reconstruction error by component count
reconstruction_errors = []
pca_models = {}
```

```
for k in range(1, max_components + 1):
    model = PCA(n_components=k, svd_solver="full")
    scores = model.fit_transform(X_clean)
    reconstructed = model.inverse_transform(scores)
    mse = np.mean((X_clean - reconstructed) ** 2)
    reconstruction_errors.append(mse)
    pca_models[k] = model
error_table = pd.DataFrame({
    "Components": range(1, max_components + 1),
    "Reconstruction MSE": reconstruction_errors})
print("\nRECONSTRUCTION ERROR")
print(error_table.round(6).to_string(index=False))
plt.figure(figsize=(7, 4))
plt.plot(range(1, max_components + 1),
         reconstruction_errors, marker="o")
plt.xlabel("Number of components")
plt.ylabel("Reconstruction MSE")
plt.title("PPCA/PCA Reconstruction Error")
plt.grid(True)
plt.show()
# 4. Reconstruct data using two principal components
model = pca_models[2]
scores = model.transform(X_clean)
reconstructed = model.inverse_transform(scores)
print("\nOriginal first sensor reading:")
print(np.round(X_clean[0], 3))
print("Reconstructed with 2 components:")
print(np.round(reconstructed[0], 3))
# 5. Generate new synthetic observations from fitted PPCA
# Covariance model: W W^T + sigma^2 I
```

```
# Use principal directions with variance above the estimated noise.
eigenvalues = ppca.explained_variance_
components = ppca.components_
signal_variances = np.maximum(eigenvalues - sigma2, 0)
W_est = components.T * np.sqrt(signal_variances)
n_new = 5
z_new = np.random.randn(n_new, max_components)
new_samples = (
    z_new @ W_est.T
    + ppca.mean_
    + np.random.normal(0, np.sqrt(max(sigma2, 0)), (n_new, n_features)))
print("\nFive generated sensor samples:")
print(pd.DataFrame(new_samples, columns=columns).round(2).to_string(index=False))
```

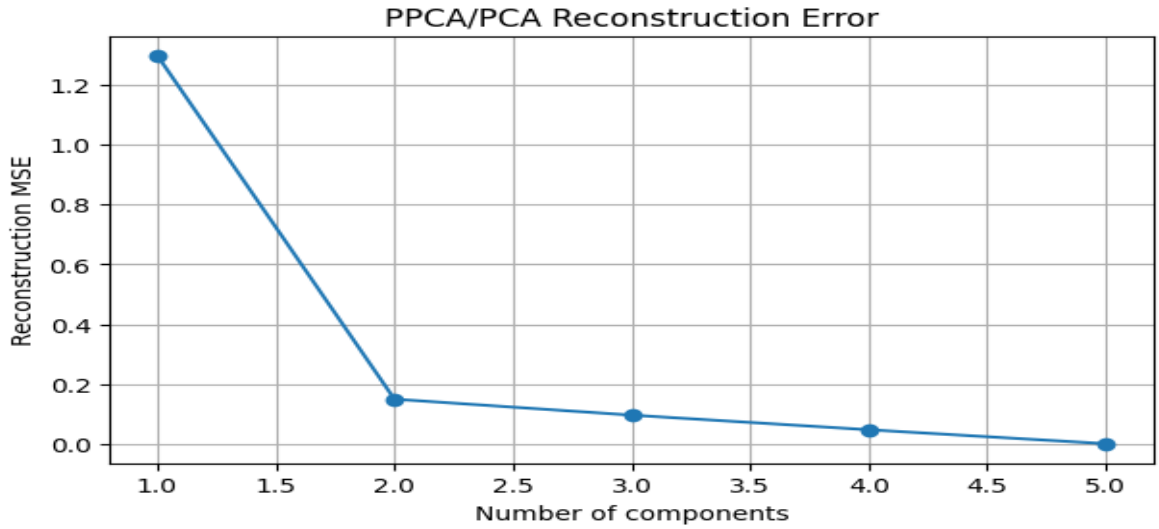
Code:

```
Sensor data shape: (600, 5)
Missing values after imputation: 0

First five sensor readings:
   Temperature  Vibration  Pressure  Current  Humidity
0        26.03      40.49     99.86    12.30     50.63
1        26.62      41.08    102.47    13.39     50.30
2        23.86      39.23     99.20    12.07     49.81
3        28.94      42.26    101.68    13.13     52.04
4        23.58      39.61    101.25    12.67     49.00

PPCA NOISE ESTIMATE
Estimated noise variance: 0.0
True generated noise variance: 0.25
RECONSTRUCTION ERROR
 Components  Reconstruction MSE
```

1	1.299365
2	0.149069
3	0.095639
4	0.046697
5	0.000000



Original first sensor reading:

[26.028 40.489 99.862 12.304 50.632]

Reconstructed with 2 components:

[25.963 40.691 99.93 12.098 50.538]

Five generated sensor samples:

Temperature	Vibration	Pressure	Current	Humidity
26.17	40.52	97.87	11.62	51.27
24.37	38.11	99.91	12.99	48.80
24.67	40.48	98.49	11.45	49.88
25.91	40.03	103.75	14.12	47.97
20.54	36.04	100.30	10.71	47.44

Result:
A PCA-based probabilistic modelling approach was applied to sensor data to estimate noise, reconstruct observations, and generate new samples.